

LLD Case Study 1: Distributed Job Scheduler

Target Patterns: Strategy Pattern, Command Pattern, Observer Pattern

Key Engineering Focus: Thread-safe task execution, pluggable scheduling algorithms, worker coordination, and lifecycle event streaming.

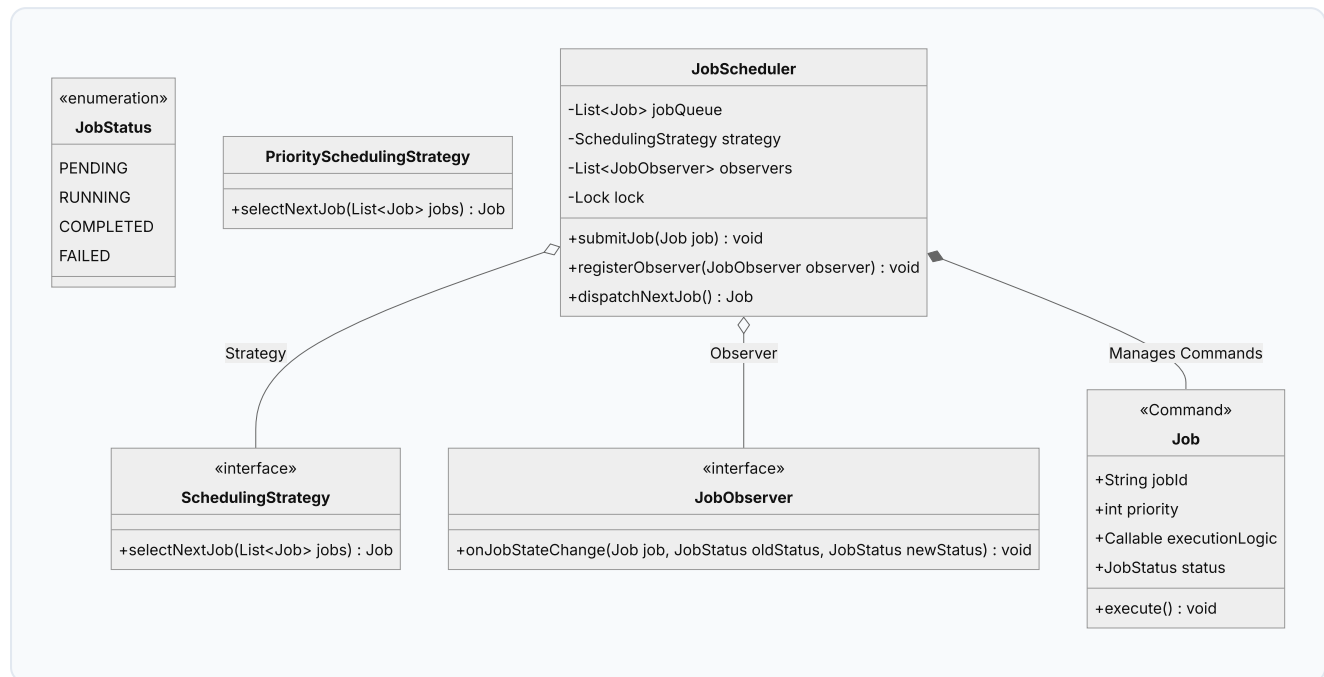
1. Problem Statement & Functional Requirements

Design a modular in-process Job Scheduler capable of scheduling, queuing, executing, and monitoring tasks across worker pools.

Requirements:

- Job Definition (Command Pattern):** Encapsulate jobs as executable units of work with retry counts and parameters.
 - Scheduling Policy (Strategy Pattern):** Support pluggable scheduling algorithms (e.g. First-Come-First-Served / Priority Scheduling vs Shortest Job First).
 - Lifecycle Notifications (Observer Pattern):** Allow external monitoring systems (logging, metrics, alerting) to subscribe to job state changes (`PENDING` , `RUNNING` , `COMPLETED` , `FAILED`).
 - Thread-Safety:** Multiple threads must be able to submit and execute jobs concurrently without state corruption.
-

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation



4. Edge Cases, Tests and Extensions

Behaviour worth knowing before an interviewer asks

Case	What happens	Why it matters
Equal priorities	<code>max(...)</code> returns the first maximal element, so ties run first-in-first-out	A free fairness guarantee, but it depends on <code>max</code> 's behaviour: test it
Low priority under constant high-priority load	Starves forever	Needs aging or a weighted fair strategy
Observer that calls <code>submit_job</code>	Deadlocks: observers run while holding a non-reentrant <code>Lock</code>	Notify outside the lock, or use <code>RLock</code>
Job raises	<code>FAILED</code> with <code>error</code> set; the scheduler keeps running	Correct isolation, but no retry
Selection cost	<code>max</code> plus <code>remove</code> is $O(n)$ per job	A heap makes it $O(\log n)$
No worker threads	<code>run_next_job()</code> runs on the caller's thread	A real scheduler adds a worker pool and a shutdown protocol

Tests, including the observer deadlock

This block extends the implementation above.

Extensions interviewers ask for

- Retries with backoff:** add `max_attempts` and `next_run_at` to `Job`; on failure re-enqueue with `next_run_at = now + base * 2**attempt` and make `select_next` skip jobs that are not yet due (the delay queue from the Python exercises).
- Worker pool:** start N threads that loop on `run_next_job`, wait on a `Condition` when the queue is empty, and stop on a sentinel.
- Scheduled and recurring jobs:** a min-heap keyed by `next_run_at`; recurring jobs re-insert themselves after each run.
- Distributed version:** the queue becomes a database table or broker; claiming a job must be atomic (`UPDATE ... WHERE status='PENDING' ... RETURNING`, or `SELECT ... FOR UPDATE SKIP LOCKED`), and a lease or heartbeat lets another worker take over a job whose owner died.

Follow-up questions

- *Why execute the job outside the lock?* A slow job would otherwise block every submit; the lock protects queue structure, not job execution.
- *At-least-once or exactly-once?* With leases and retries it is at-least-once, so jobs must be idempotent; exactly-once needs an idempotency key stored with the result.
- *How do you stop a runaway job?* Cooperative cancellation with a flag the job polls, or run jobs in separate processes you can terminate; Python threads cannot be killed safely.